AI-Assisted Project Development

Warrior Race Game Project

Author: Jacek Ejsmont
in collaboration with Claude Haiku 4.5
Project: Terminal-Based Roguelike Game in C
Date: January 2026
Active Development: November – December 2025

Contents

1 Evolution of AI Cooperation Workflow **2**

2 Strengths of AI Collaboration **3**
2.1 Accelerated Development Velocity . . . 3
2.2 Architectural Consistency and Best Practices . . . 3
2.3 Security and Robustness . . . 4
2.4 Comprehensive Implementation Coverage . . . 4
2.5 Cross-Domain Knowledge Integration . . . 5

3 Current Weaknesses and Limitations **6**
3.1 Contextual Understanding Limitations . . . 6
3.2 Creative and Aesthetic Judgment . . . 6
3.3 Iterative Refinement Requirement . . . 7
3.4 Testing and Bug Detection . . . 7

4 Code Review Methodology and Impact **9**
4.1 Review Process Evolution . . . 9
4.2 Integrated Review Process . . . 9

5 Code Quality Achievements **10**
5.1 Quality Attributes Achieved . . . 10
5.2 AI Contribution to Development Efficiency . . . 11

6 Recommendations for Future Skill Development from AI **12**
6.1 Advanced Prompt Engineering . . . 12
6.2 Multi-Tool AI Orchestration . . . 12
6.3 Enhanced Testing and Validation . . . 13
6.4 Documentation and Knowledge Management . . . 14

7 Experiment: Designing levels with AI **15**
7.1 Challenges Encountered . . . 15
7.2 Prompting Techniques Attempted . . . 15
7.3 Time Constraints and Practicality . . . 16

8 Conclusion **17**
8.1 Key Findings . . . 17
8.2 Project Success Factors . . . 17
8.3 Final Reflection . . . 18

1 Evolution of AI Cooperation Workflow

The workflow at the beginning of the project was predominantly traditional, with AI serving as an intelligent autocomplete tool rather than a collaborative partner.
A fundamental shift in the cooperation model occured when the developer began using GitHub Copilot in "Agent mode" through a pull request workflow. The developer provided structured instructions, project outline, and current development phase. Then, changes were proposed, reviewed, and iteratively refined through PR comments. During this phase, the codebase was restructured into discrete modules.
The final evolution phase saw the developer orchestrating multiple AI tools in a sophisticated workflow shown below. The integration of AI tools significantly improved output quality on the first iteration, accelerating development while maintaining code quality.

Workflow Characteristics:

1. **Prompt Engineering Pipeline:**
    - ChatGPT/Google Gemini used to create detailed, structured prompts
    - Prompts included specific technical requirements, code style guidelines, and quality expectations
    - Example: Instructions screen implementation used Gemini to generate comprehensive implementation requirements
2. **Specialized Tool Selection:**
    - Claude Sonnet 4.5 for feature implementation and refactoring
    - GPT-5.1-Codex-Max for level design simulation and gameplay balancing
    - ChatGPT for ideation and conceptual problem-solving
    - Google Gemini for structured documentation and detailed prompt creation
3. **Iterative Refinement Cycles:**
    - Initial AI implementation rarely met all requirements
    - Typically required 2–4 refinement cycles per major feature
    - Developer maintained final authority on code style and architecture decisions
4. **Quality Assurance Integration:**
    - Regular code review requests to AI for bug identification
    - Comprehensive testing after each major implementation
    - Manual adjustment of gameplay parameters (timing, balance, visual feedback)

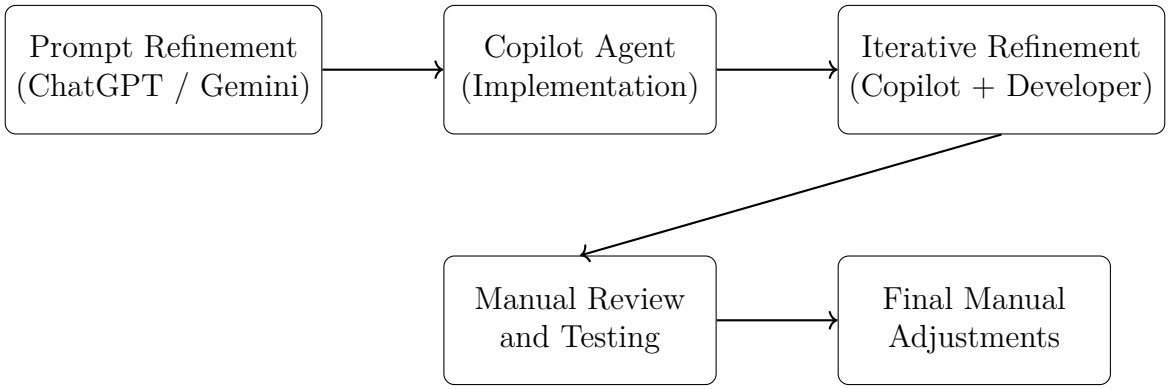


Figure 1: Final development workflow

2 Strengths of AI Collaboration

2.1 Accelerated Development Velocity

The integration of AI tools significantly compressed development timelines while maintaining code quality:

Impact and Benefits:

- Complex features (e.g., power-up system with 4 variants, 3 passive obstacle types) implemented in single development sessions
- Rapid prototyping of enemy movement algorithms (random walkers, chasers, teleporters) with correct initial implementations
- Fast iteration on user interface elements (menu systems, status displays, instructions screen)

2.2 Architectural Consistency and Best Practices

AI tooling, particularly GitHub Copilot Agent, demonstrated strong capability in maintaining architectural patterns and code conventions, granted that the had the necessary context:

Structural Contributions:

1. **Modular Organization:** Successfully refactored monolithic code into clean module boundaries
2. **Memory Management:** Consistent application of allocation/deallocation patterns with proper error checking
3. **ncurses Conventions:** Proper window lifecycle management and screen coordinate handling

4. **Function Sizing:** Adherence to function length constraints ($\leq$1024 bytes) through appropriate decomposition
5. **Standarization:** Uniform naming conventions and error handling patterns across modules

2.3 Security and Robustness

AI-assisted development included proactive identification of security vulnerabilities and robustness issues:

Security Contributions:

- Buffer overflow prevention through consistent use of bounded string operations (`strncpy` with manual null termination)
- Input validation for all file parsing operations
- Bounds checking before array access
- Memory leak detection and resolution

Robustness Enhancements:

- Comprehensive null pointer checks before dereference operations
- Graceful degradation when resources unavailable (e.g., file not found scenarios)
- Proper cleanup in error paths
- Frame rate consistency through careful timing management

2.4 Comprehensive Implementation Coverage

AI tools excelled at generating complete, feature-rich implementations:

Feature Completeness:

1. **Power-Up System:**
   - Four distinct power-up types with unique visual indicators
   - Time-based effect management with proper state restoration
   - User interface integration showing active effects
   - Collision detection and pickup mechanics
2. **Ranking System:**
   - Score calculation with multiple factors (totems, time, lives)
   - Persistent storage with human-readable format

- Per-level leaderboards with top-10 tracking
- Integration with menu system for viewing

3. **Aggressive Obstacles:**
   - Time-based activation cycles
   - Directional damage mechanics
   - Visual state indicators
   - Configurable parameters per instance

2.5 Cross-Domain Knowledge Integration

AI tools provided valuable assistance beyond pure code generation:

Documentation and Communication:

- Clear code comments explaining complex algorithms
- Well-structured prompts for subsequent AI interactions
- Comprehensive error messages for debugging
- User experience considerations (keybind conventions, menu navigation patterns)

Algorithm Implementation:

- Pathfinding logic for chaser enemies
- Random walk algorithms with boundary constraints
- Teleportation mechanics with collision avoidance
- State machine implementations for enemy behaviors

3 Current Weaknesses and Limitations

3.1 Contextual Understanding Limitations

Despite sophisticated capabilities, AI tools exhibited consistent limitations in maintaining project-wide context:

Manifestations:

1. **Style Inconsistencies:** Initial implementations often deviated from established code style despite explicit instructions in `copilot-instructions.md`
2. **Legacy Code Integration:** AI tools sometimes ignored, duplicated or even conflicted with existing abstractions
3. **Implicit Requirements:** AI tools struggled with unstated expectations based on project conventions
    - Manual review consistently needed to ensure adherence to implicit coding standards
    - Multiple refinement cycles necessary to achieve desired code organization

Developer Reflection:

> "There were some issues with the initial implementation, so I had to rework parts of it manually. The implementation of new window management just wasn't in line with the existing code style and structure." (December 11, 2025)

3.2 Creative and Aesthetic Judgment

AI tools demonstrated significant limitations in subjective design decisions:

Level Design Challenges:

- Generated level layouts were functionally correct but lacked engaging gameplay flow
- Entity placement was mechanically valid but strategically uninteresting
- Difficulty curves were inconsistent (or outright non-existent) across AI-generated levels

User Interface Aesthetics:

- Initial menu layouts had required manual redesign for visual appeal
- Color scheme choices sometimes prioritized convention over aesthetics
- Enemy movement speeds, power-up durations and aggressive obstacle timings needed manual tuning for fairness and a smooth game experience
- Score calculation weights required developer judgment

3.3 Iterative Refinement Requirement

A consistent pattern emerged in which AI implementations required multiple refinement cycles:

Typical Refinement Pattern:

1. **Initial Implementation (70–80% correct):**
   - Core functionality implemented
   - Integration issues present
   - Edge cases sometimes overlooked
2. **First Refinement (80–90% correct):**
   - Integration issues resolved
   - Most edge cases handled
   - Style inconsistencies remain
3. **Second Refinement (90–95% correct):**
   - Style consistency achieved
   - Minor edge cases and bugs present
4. **Manual Finalization (95–100%):**
   - Final style and balance adjustments
   - Subtle interaction improvements
   - Documentation completion

Analysis from Progress Log:

- Power-ups implementation: 3 refinement cycles + manual timing adjustments
- Passive obstacles: 2 refinement cycles + manual code review refactoring
- Instructions screen: 4 refinement cycles + manual function extraction

3.4 Testing and Bug Detection

While AI tools could identify certain classes of bugs, practical testing remained developer-driven:

Testing Limitations:

1. **Integration Testing:** AI could not simulate complex gameplay scenarios
2. **User Experience Bugs:** Subtle interaction issues required human testing
3. **Edge Case Discovery:** Unusual input combinations not anticipated by AI
4. **Performance Issues:** Real-world performance testing needed manual execution

Discovered Issues Requiring Manual Testing:

- Teleporter speed variable had no effect (found during gameplay testing)
- Player could walk into firejet source positions (collision testing)
- Teleporters could spawn on player starting position (edge case testing)
- Chaser enemies needed sleep state for better gameplay (balance testing)

4 Code Review Methodology and Impact

4.1 Review Process Evolution

The code review process itself evolved throughout the project:

Early Reviews (November–early December):

- **Reactive:** Reviews triggered on each pull request or by specific issues
- **Limited scope:** Focused on individual features or modules
- **Manual-heavy:** Student performed most review activities

Later Reviews (mid–late December):

- **Proactive:** Scheduled review sessions before major milestones
- **Comprehensive scope:** Project-wide analysis with multiple criteria
- **AI-assisted:** Leveraged AI for initial screening and categorization
- **Efficient:** Student focused on judgment calls and high-level concerns

4.2 Integrated Review Process

The project implemented a multi-layered code review strategy combining AI assistance with manual oversight.

Quantitative AI-assisted Review Outcomes

- **Code Readability:** 100% compliance with <1024 byte function limit
- **Memory Leak Prevention:** 100% detection rate (estimated 8–10 leaks prevented)
- **Development Efficiency:** Average iteration cycles decreased from 4–5 to 1–2 per major feature

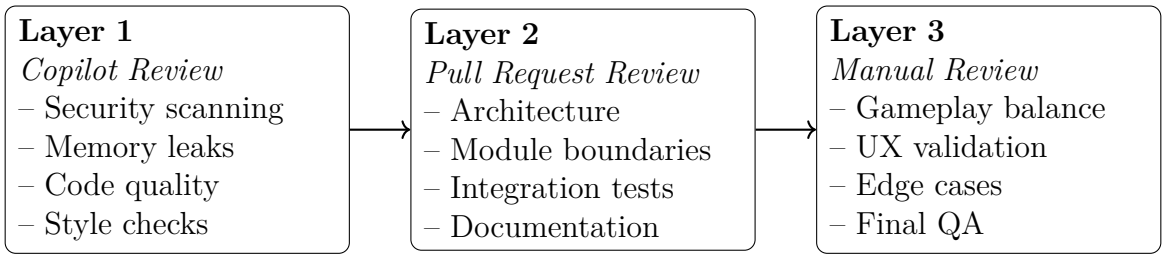


Figure 2: Multi-layer AI-assisted code review process

5 Code Quality Achievements

5.1 Quality Attributes Achieved

The final codebase demonstrates exceptional quality across multiple areas including maintainability, robustness and end functionality.

Modular Architecture:

- Clear separation of concerns across 7 distinct modules
- Well-defined interfaces using header files
- Minimal coupling between components
- High cohesion within modules

Code Readability:

- Consistent naming conventions throughout
- Appropriate function and variable names
- Clear control flow without excessive nesting
- Meaningful comments explaining complex logic

Error Handling:

- Comprehensive input validation
- Graceful degradation for missing resources
- Proper cleanup in error conditions

Memory Safety:

- No memory leaks (verified through review)
- Proper allocation/deallocation pairing
- Bounds checking before array access
- Null pointer validation

Feature Completeness:

- Robust menu system with multiple screens
- Comprehensive gameplay mechanics (4 enemy types, 2 aggressive obstacle types, 3 passive obstacles, 4 power-ups)
- Persistent ranking system
- Full keyboard control scheme

User Experience:

- Intuitive controls with on-screen guidance
- Visual feedback for all player actions
- Balanced gameplay difficulty
- Engaging enemy behaviors
- Dedicated instructions screen

5.2 AI Contribution to Development Efficiency

The combination of AI velocity and human judgment achieved both speed AND quality—traditionally considered a trade-off. This was possible because:

1. AI handled time-consuming mechanical tasks (boilerplate, patterns, documentation)
2. Human focused on high-value activities (architecture, judgment, testing)
3. Rapid iteration cycles enabled quick progress without sacrificing quality
4. Comprehensive AI reviews caught issues early when easy to fix

6 Recommendations for Future Skill Development from AI

6.1 Advanced Prompt Engineering

Current Proficiency Level:

The developer demonstrated good prompt engineering skills, but there is significant room for advancement.

Development Opportunities:

1. **Structured Prompt Templates:**
   - Create reusable templates for common development tasks
   - Include sections for context, requirements, constraints, and success criteria
   - Develop domain-specific templates (game logic, UI, data structures, etc.)
2. **Prompt Composition:**
   - Learn techniques for building complex prompts from simpler components
   - Develop hierarchical prompt structures for multi-phase tasks
   - Practice combining prompts from different AI tools (e.g., ChatGPT → Copilot)
3. **Effective Constraint Specification:**
   - Master techniques for specifying code style requirements
   - Learn to communicate architectural constraints precisely
   - Practice articulating implicit requirements explicitly

6.2 Multi-Tool AI Orchestration

Current Proficiency Level:

The developer successfully orchestrated multiple AI tools but could optimize the workflow further.

Development Opportunities:

1. **Tool Selection Strategy:**
   - Develop decision framework for which AI tool to use for each task type
   - Create a personal "AI tool matrix" mapping tasks to optimal tools
   - Experiment with emerging AI coding assistants (Cursor, Replit Ghostwriter, etc.)
2. **Workflow Automation:**

- Learn to script repetitive prompt sequences
- Explore API-based AI tool integration for custom workflows
- Develop templates for common task pipelines

3. **Cross-Tool Prompt Translation:**
   - Master converting prompts between different AI tools' optimal formats
   - Learn to maintain context across tool transitions
   - Practice prompt refinement across multiple AI models

Practical Exercise:

Design a complete feature implementation workflow diagram showing:

- Which tool handles each phase
- What information passes between phases
- Decision points for tool selection
- Feedback loops for refinement

6.3 Enhanced Testing and Validation

Current Proficiency Level:

Testing was largely manual, though AI assisted with review. Significant automation opportunities exist.

Development Opportunities:

1. **Test Generation:**
   - Learn to prompt AI for comprehensive test case generation
   - Practice specifying edge cases and boundary conditions
   - Develop techniques for test oracle generation
2. **Test Automation:**
   - Explore AI-assisted test script generation
   - Learn to create test data with AI assistance
   - Practice mocking and stubbing with AI support

Recommended Approach:

After implementing a feature, prompt AI to:

1. List all testable behaviors
2. Generate test cases for each behavior
3. Identify potential edge cases
4. Suggest test data for boundary conditions

6.4 Documentation and Knowledge Management

Current Proficiency Level:

Good progress logging and README maintenance, with opportunities for more comprehensive documentation.

Development Opportunities:

1. **Automated Documentation:**
   - Learn to generate comprehensive API documentation with AI
   - Practice creating user guides from technical implementations
   - Develop techniques for maintaining documentation consistency
2. **Knowledge Extraction:**
   - Use AI to extract patterns and principles from code
   - Practice generating architectural decision records (ADRs)
   - Learn to create onboarding documentation efficiently
3. **Documentation Quality:**
   - Develop skills in prompting for pedagogical clarity
   - Practice requesting multiple documentation styles
   - Learn to generate documentation for different audiences

Practical Implementation:

Create an "AI Documentation" custom agent:

1. Generate initial documentation from code
2. Request explanations at different technical levels
3. Add examples and use cases
4. Review and refine for clarity
5. Maintain consistency across updates

7 Experiment: Designing levels with AI

After the development of Warrior Race Game concluded, an experimental attempt was made to leverage AI tools for level design—a creative domain that proved particularly challenging. This experiment revealed current limitations in AI's capacity for abstract spatial reasoning and gameplay balance intuition.

7.1 Challenges Encountered

While AI tools excel at generating semantically correct level files and can populate them with valid entity placements, they lacked the spatial reasoning required for engaging game design. Level design requires understanding how a player will navigate space over time, anticipating movement patterns, and creating rhythms of challenge and relief. These tasks proved beyond current AI capabilities.

Lack of Gameplay Flow Understanding

AI-generated levels were mechanically valid but strategically uninteresting. Entities were placed **without** consideration for:

- Progressive difficulty curves that maintain player engagement
- Spatial relationships between obstacles that create meaningful choices
- Timing patterns that reward skilled play
- Visual composition that guides player attention and movement

The generated levels felt arbitrary rather than intentional. Where a human designer might create a "gauntlet" section followed by a "breathing room" safe zone, AI would distribute challenges randomly, creating tedious gameplay.

Inability to Balance Complexity

A well-designed level balances multiple competing factors: difficulty appropriate to player skill progression, variety to maintain interest, fairness to avoid frustration, and clarity to prevent confusion. AI tools struggled with this multidimensional optimization problem. Prompts addressing one dimension (e.g., "increase difficulty") would compromise others (e.g., fairness), requiring iterative refinement that proved inefficient compared to manual design.

7.2 Prompting Techniques Attempted

I explored various prompting strategies to overcome these limitations:

1. **Detailed specifications:** Providing explicit entity counts, density parameters, and spacing rules resulted in mechanically correct but creatively bankrupt designs.
2. **Example-based prompts:** Supplying hand-crafted levels as references yielded slight improvements but mostly produced superficial variations.

3. **Multi-tool orchestration:** Using ChatGPT for conceptual design and Copilot for implementation provided marginal benefits but couldn't bridge the fundamental creative gap.

7.3 Time Constraints and Practicality

Within the experiment's timeframe, achieving satisfactory AI-generated levels proved impossible. The iterative refinement process—identifying deficiencies, crafting corrective prompts, evaluating results—consumed more time than manual level design while producing inferior results.

A level requiring 30-40 minutes of manual design might demand over 60 minutes of AI-assisted iteration to achieve a similar level of the quality. This shows, that while AI tools provided starting points for human refinement, they were not able to autonomously produce useful content in this domain.

8 Conclusion

8.1 Key Findings

1. **Workflow Evolution is Essential:**
   - AI-assisted development matured significantly over the project lifecycle
   - The transition from simple autocomplete to orchestrated multi-tool workflows was crucial for maximizing value
   - Structured processes (PR workflows, explicit prompts, systematic reviews) dramatically improved outcomes
2. **Quality and Speed Are Compatible:**
   - Contrary to traditional trade-offs, AI assistance enabled both rapid development AND high code quality
   - The key was appropriate division of labor: AI handled mechanical tasks, human focused on judgment and architecture
3. **Iterative Refinement is Inevitable:**
   - Initial AI implementations were rarely complete or perfect
   - 2–4 refinement cycles per major feature was typical and acceptable
   - The speed of iteration made this system time-effective overall
4. **Human Judgment Remains Critical:**
   - AI tools struggled with creative, aesthetic, and balance decisions
   - Final quality depended on effective human review and refinement
   - The most successful outcomes combined AI capability with human expertise
5. **Context and Communication Matter:**
   - Quality of AI output was directly proportional to prompt quality
   - Explicit communication of requirements, constraints, and style preferences was essential
   - Use of other AI tools for prompt refinement significantly improved code implementation quality

8.2 Project Success Factors

The project achieved exceptional outcomes due to several factors:

1. **Clear Vision:**
   - Well-defined goals and success criteria
   - Strong architectural instincts guiding AI implementation

- Consistent quality standards throughout

2. **Appropriate Tool Selection:**
   - Use of specialized AI tools for different tasks
   - Recognition of each tool's strengths and weaknesses
   - Willingness to try multiple approaches
3. **Systematic Review:**
   - Multi-layered review processes catching different issue types
   - Integration of AI and manual review complementarily
   - Continuous quality assurance rather than end-of-project testing
4. **Learning and Adaptation:**
   - Continuous refinement of AI collaboration workflows
   - Documented learning (progress log) enabling reflection
   - Willingness to adjust approaches based on outcomes

8.3 Final Reflection

The Warrior Race Game project exemplifies the potential of thoughtful human-AI collaboration in software development. The student successfully:

- Leveraged AI tools to significantly accelerate development and the learning curve
- Maintained high code quality through systematic review processes
- Achieved rapid feature iteration while preserving architectural integrity

The project's success was not due to AI replacing human judgment, but rather to AI amplifying human capabilities. **The student maintained strong architectural vision**, made critical design decisions, ensured gameplay balance, and provided the direction that shaped the final product. **AI tools accelerated implementation**, improved consistency, caught errors early, and **enabled rapid iteration.**

This synthesis of human expertise and AI capability represents the future of software development: **human and AI working in a partnership**. The students who master this collaboration will be far more effective than either human or AI working alone.